

NAPCORE Helpdesk: From Search To Grounded Chat Helpdesk

Presentation Draft

NAPCORE Helpdesk Team

2026-03-28

Problem And Goal

- ▶ Need a trusted answer layer for multimodal standards implementation.
- ▶ Main risk is fast but uncited guidance that creates delivery risk.
- ▶ Goal is practical answers with traceable references and controlled escalation.

System Context (C4 Level 1)

Five stakeholder groups share one helpdesk interface:

- ▶ PTO (Public Transport Operator) — responsible for daily service delivery
- ▶ PTA (Public Transport Authority) — sets policy and compliance standards
- ▶ Developer — implements integrations and APIs
- ▶ ITS System Integrator — connects helpdesk to broader mobility ecosystem
- ▶ Ticketing Agent — manages passenger-facing systems

External systems:

- ▶ Approved GitHub repositories (NeTEx-CEN/NeTEx) — knowledge source
- ▶ LLM provider — grounded generation only; no unconstrained model memory

Key architectural constraint:

- ▶ All answers sourced from approved repositories only

Scope

- ▶ Standards: Transmodel, NeTEx, SIRI, OJP/OpRa, DATEX II.
- ▶ Priority source: NeTEx-CEN/NeTEx and approved companion repositories.
- ▶ User channel: one operator console with a chat-style helpdesk experience.

Evolution Of Helpdesk Implementations

- ▶ Stage 1: keyword search over standards documents.
- ▶ Stage 2: grounded Q and A with FAQ-first orchestration.
- ▶ Stage 3: editorial workflow, routing, and KPI visibility.
- ▶ Stage 4: chat-style sessions ready for deterministic and LLM-backed generation.
- ▶ The trust model stays fixed: allow-listed repositories, citations, and review gates.

Why The Evolution Matters

- ▶ Search helps lookup, but not decision-ready guidance.
- ▶ FAQ-first gives low-latency canonical answers for recurring issues.
- ▶ Editorial workflow converts runtime demand into governed knowledge.
- ▶ Chat sessions improve usability without weakening provenance.
- ▶ Companion repositories expand coverage without relaxing the trust boundary.

Core Approach

- ▶ FAQ-first with RAG fallback.¹
- ▶ FAQ-first: return approved answers when confidence is high.
- ▶ RAG fallback: retrieve source chunks, then generate a grounded answer.
- ▶ Generation profiles: deterministic-grounded now, LLM-ready when configured.
- ▶ If evidence is weak, abstain explicitly.

¹RAG fallback means that when no high-confidence FAQ answer is available, the system retrieves relevant source chunks and then generates an answer grounded in that retrieved evidence.

Trust And Safety

- ▶ Repository allowlist enforces the knowledge boundary.
- ▶ Generation is limited to retrieved evidence.
- ▶ Citation gate blocks uncited publication.
- ▶ Editorial gate controls draft, review, approve, and publish states.

Functional Requirements Highlights

- ▶ FR-001: question answering through the helpdesk UI.
- ▶ FR-004: ingestion and indexing of approved GitHub sources.
- ▶ FR-005: retrieval and grounded generation.
- ▶ FR-006: anti-hallucination and boundary enforcement.
- ▶ FR-002 and FR-003: editorial curation and FAQ promotion.

Data Model Logic

- ▶ Source governance: approved_repositories -> source_documents -> source_chunks.
- ▶ FAQ lifecycle: faq_entries -> faq_versions -> review_events.
- ▶ Runtime telemetry: question_events -> retrieval_events.
- ▶ Evidence mapping: answer_evidence_links ties answers to source chunks.

Why Hallucinations Are Prevented

- ▶ Only approved repositories are retrievable.
- ▶ Unsupported claims fail citation and support checks.
- ▶ Evidence links provide claim-to-source traceability.
- ▶ Unknown questions produce abstention, not speculation.

User Experience Flow

- ▶ User opens chat session or operator console and submits question.
- ▶ System attempts FAQ match first.
- ▶ On miss or low confidence, the system executes retrieval plus grounded generation.
- ▶ Response includes references, confidence, and request trace.
- ▶ Session history supports iterative questioning without losing provenance.
- ▶ Feedback and editorial routing support continuous improvement.

Current Product State

- ▶ Frontend now includes a chat-style session UX alongside the editorial console.
- ▶ Backend supports deterministic grounded generation and an optional LLM-ready mode.
- ▶ Knowledge index is built from approved repositories and selected companion repositories.
- ▶ Editorial board, KPI metrics, and promotion candidate flows remain part of the same product.

Current Chat UX

- ▶ Live chat session view with per-turn evidence, confidence, and request trace.
- ▶ Same surface supports deterministic grounded answers today and LLM-ready mode later.

Chat Session

Multi-turn UX with per-turn evidence, trace IDs, and session continuity. LLM-ready profile can be enabled in backend later.

Session ID

sess-local

User ID

user-local

Generation Profile

deterministic-grounded (active) ▼

USER

How should a NeTeX timetable exchange be validated before publication?

ASSISTANT

For NeTeX timetable exchange, start by defining ServiceFrame and TimetableFrame, then validate profile constraints before publishing.

mode: faq · confidence: 0.93 · reviewRequired: false

- [NeTeX repository overview](#) · README.md · placebo

requestId: req-mnbtwpqx-6d7svj

Message

Ask a standards question...

☒ apply selected standards scope (NeTeX)

Send

New Session

Operations And Governance

- ▶ Reviewer decisions are logged and auditable.
- ▶ Editorial operations run through Django Admin.
- ▶ Recurring intents are promoted into a curated FAQ backlog.
- ▶ Re-indexing handles source and standards updates.
- ▶ Metrics monitor quality and safety continuously.

MVP Success Metrics

- ▶ Citation coverage for published FAQs: 100%.
- ▶ Unsupported-claim publication rate: 0%.
- ▶ P95 latency: ≤ 8 seconds.
- ▶ Retrieval success on covered intents: $\geq 90\%$.

Current Artifacts

- ▶ Functional requirements, testing pack, and local run guide.
- ▶ RAG architecture, database logic, and updated C4 architecture documents.
- ▶ Technology baseline: Django 5 + DRF backend, React/Vite frontend, editorial workflow.
- ▶ Chat session UX, deterministic grounded answering, and LLM-ready generation adapter.
- ▶ PlantUML and exported SVG architecture assets for reproducible documentation.

Next Steps

1. Complete pilot testing across chat UX, editorial workflow, and promotion flows.
2. Enable and evaluate grounded LLM mode against deterministic fallback behavior.
3. Refine ingestion profiles for companion repositories and standards-specific scopes.
4. Use pilot evidence to decide production hardening and deployment model.

Appendix: Example Questions

- ▶ How to use NeTEx for exchanging a timetable?
- ▶ How to implement IDs for a Stop Place registry?

Expected outputs: - practical implementation guidance, - explicit citations, - abstention where evidence is insufficient.